Programming Techniques 2026–2027

Lecture 6: Derived Types, Linked Lists, and Custom Operators

Hannu Parviainen

Universidad de la Laguna

October 13, 2026

Derived Types

Derived Types

Example: 3D vector type

```
module geometry
  implicit none

  type :: vector3d
    real :: x, y, z
  end type vector3d

end module geometry
```

- Basic Fortran types can be combined to create more complex *derived* types.
- Derived type definitions should be placed in a module.

Example: type usage

```
program test_geometry
  use geometry
  implicit none
  type(vector3d) :: a
  a = vector3d(0.0, 1.0, 0.0)
  print *, a
end program test_geometry
```

- Variables are declared with `type(name)`.
- A value is built with a constructor, which takes the components in the order of their definition.

Nested derived types

Previously defined types can be used as components of other derived types.

Example: sphere

```
module geometry
  implicit none

  type :: vector3d
    real :: x, y, z
  end type vector3d

  type :: sphere
    type(vector3d) :: centre
    real :: radius
  end type sphere
end module geometry
```

Example: sphere usage

```
program test_geometry
  use geometry
  implicit none
  type(sphere) :: s
  s = sphere(vector3d(0.0, 1.0, 0.0), 5.0)
  print *, s
end program test_geometry
```

Derived type assignment

Ways to assign

- Component-by-component using %.
- As a whole object using a constructor.
- Assignment between two objects of the same derived type is intrinsic.

Inside a constructor, a derived-type component such as `centre` is given as a value of its own type: a variable or another constructor.

```
type(vector3d) :: pt1
type(sphere) :: s, ball

! component by component
pt1%x = 1.0
s%radius = 3.0
s%centre%x = 1.0

! whole object with constructors
pt1 = vector3d(1.0, 2.0, 3.0)
s%centre = vector3d(1.0, 2.0, 3.0)
s = sphere(s%centre, 10.0)
s = sphere(vector3d(1.0, 2.0, 3.0), 10.0)

! intrinsic assignment between objects
ball = s
```

Allocatable and pointer components

- A component can be allocatable (since Fortran 2003) or a pointer.
- Prefer allocatable components. They are freed automatically, and assignment copies their data (a *deep copy*).
- A pointer component is only pointer-assigned in a copy, so both objects then share one target.
- Use pointer components for linked structures, such as a list node that points to the next node.

```
module spectra
  implicit none
  type :: spectrum
    character(len=:), allocatable :: name
    real, allocatable :: flux(:)
  end type spectrum
end module spectra

program components
  use spectra
  implicit none
  type(spectrum) :: a, b
  a = spectrum('Vega', [1.0, 2.0, 3.0])
  b = a                    ! deep copy
  b%flux(1) = 99.0
  print *, a%flux          ! 1.0  2.0  3.0
  print *, b%flux          ! 99.0  2.0  3.0
end program components
```

Derived type I/O

List-directed I/O

- A derived type can be printed and read with `print *` and `read *` if it has no pointer or allocatable components.
- I/O proceeds component-by-component in order.

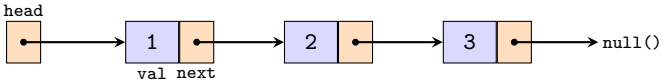
The two statements on the right print the same values.

```
print *, s

print *, s%centre%x, s%centre%y, &
         s%centre%z, s%radius
```

Linked Lists

Linked lists



- A linked list is a chain of cells. Each cell holds a value and a pointer to the next cell.
- `head` points to the first cell. The pointer of the last cell is null.
- Every cell is allocated separately, so the list grows and shrinks one cell at a time, without copying.
- The price: there is no direct access to cell number i. You have to follow the pointers.

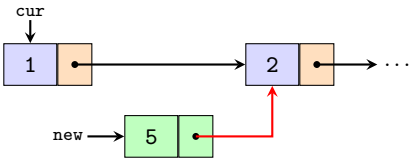
```
type :: cell
  integer :: val
  type(cell), pointer :: next => null()
end type cell
```

Adding cells at the front builds the list above:

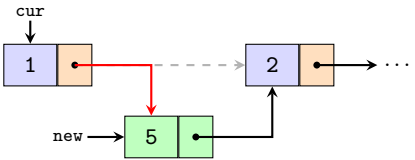
```
do i = 3, 1, -1
  allocate(new)
  new%val = i
  new%next => head
  head => new
end do
```

Linked lists: inserting a cell

1. `new%next => cur%next`



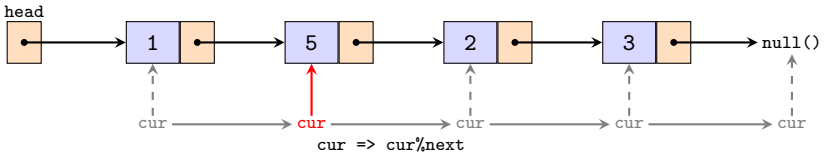
2. `cur%next => new`



- To insert a cell after the one `cur` points to, allocate it and change two pointers.
- The order matters: step 1 saves the link to the rest of the list before step 2 overwrites it. In the other order, the new cell points to itself: the rest of the list is leaked, and a walk never ends.
- No other cell is touched or moved, however long the list is.

```
cur => head
allocate(new)
new%val = 5
new%next => cur%next
cur%next => new
```

Linked lists: walking through a list



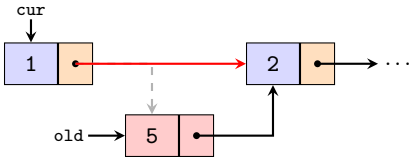
- Start with `cur` pointing at the first cell.
- `cur => cur%next` moves `cur` one cell forward.
- After the last cell, `cur` is null, `associated(cur)` is false, and the loop ends.

```
cur => head
do while (associated(cur))
  print *, cur%val
  cur => cur%next
end do
```

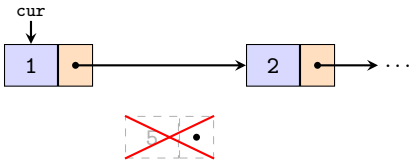
This prints 1, 5, 2, and 3.

Linked lists: removing a cell

1. `old => cur%next` **2.** `cur%next => old%next`



3. `deallocate(old)`



- Keep a pointer to the cell (`old`), let the list bypass it, and then free it.
- Without `old`, nothing would point to the cell after step 2: a memory leak.
- To free a whole list, remove the first cell until `head` is null.

```
old => cur%next
cur%next => old%next
deallocate(old)
```

```
do while (associated(head))
  old => head
  head => head%next
  deallocate(old)
end do
```

Arrays of pointers

Notes

- Fortran has no arrays of pointers: `real, pointer :: p(:)` is one pointer to an array.
- To get an array of pointers, wrap a pointer in a derived type and make an array of that type.
- You cannot reference the pointer component of the whole array in one go.
- If desired, the container array could be `allocatable`.

```
module pointers
  implicit none
  type :: iptr
    integer, pointer :: compon => null()
  end type iptr
end module pointers

program pointer_array
  use pointers
  implicit none
  type(iptr), dimension(100) :: ints
  integer, target :: k = 5

  ints(10)%compon => k          ! ok
  print *, ints(10)%compon      ! ok: 5
  ! print *, ints(:)%compon     ! not allowed
end program pointer_array
```

Custom Operators

Arithmetic with derived types

- Fortran allows you to define complex derived types.
- But using them directly for calculations leads to cumbersome code (and bugs, lots and lots of bugs).
- Here `c = s*a + b` has to be written out component by component.
- The examples in this section use a 2D vector to keep the code short.

```
module geometry
  implicit none
  type :: vector2d
     real :: x, y
  end type vector2d
end module geometry
```

```
program test
  use geometry
  implicit none
  type(vector2d) :: a, b, c
  real :: s = 2.3
  a = vector2d(0.0, 1.0)
  b = vector2d(1.0, 1.0)
  c = vector2d(s*a%x+b%x, s*a%y+b%y)
  print *, c
end program test
```

Step 1: functions

Code can be simplified (and made more robust) by using functions.

```
module geometry
  implicit none
  type :: vector2d
    real :: x, y
  end type vector2d

contains
  pure type(vector2d) function sumvv(a, b)
    type(vector2d), intent(in) :: a, b
    sumvv = vector2d(a%x + b%x, a%y + b%y)
  end function sumvv

  pure type(vector2d) function mulrv(a, b)
    real, intent(in) :: a
    type(vector2d), intent(in) :: b
    mulrv = vector2d(a*b%x, a*b%y)
  end function mulrv
end module geometry
```

```
program test
  use geometry
  implicit none
  type(vector2d) :: a, b, c
  real :: s = 2.3
  a = vector2d(0.0, 1.0)
  b = vector2d(1.0, 1.0)
  c = sumvv(mulrv(s, a), b)
  print *, c
end program test
```

Step 2: operators

```
module geometry
  implicit none
  type :: vector2d
     real :: x, y
  end type vector2d
  interface operator(+)
     module procedure sumvv
  end interface
  interface operator(*)
     module procedure mulrv
  end interface
contains
  pure type(vector2d) function sumvv(a, b)
    type(vector2d), intent(in) :: a, b
    sumvv = vector2d(a%x + b%x, a%y + b%y)
  end function sumvv
  pure type(vector2d) function mulrv(a, b)
    real, intent(in) :: a
    type(vector2d), intent(in) :: b
    mulrv = vector2d(a*b%x, a*b%y)
  end function mulrv
end module geometry
```

```
program test
  use geometry
  implicit none
  type(vector2d) :: a, b, c
  real :: s = 2.3
  a = vector2d(0.0, 1.0)
  b = vector2d(1.0, 1.0)
  c = s*a + b
  print *, c
end program test
```

And defining custom operators can make your code even cleaner: the program now says `c = s*a + b`.

How to define a custom operator

Custom operators are defined with an `interface operator` block, which should be placed in a module.

- A standard operator (+, *, ==, ...)

```
interface operator(+)
   module procedure newsum
end interface
```

- A new operator named `.op.`

```
interface operator(.op.)
   module procedure newop
end interface
```

The functions implementing the operators are defined later in the module after the `contains` keyword.

Rules for custom operators

- An operator function takes one argument (unary) or two (binary), all `intent(in)`.
- You can extend the standard operators to your own types, but you cannot change what they do for the intrinsic types.
- Extended standard operators keep their usual precedence, so `s*a + b` works as expected.
- A named binary operator such as `.dot.` has the lowest precedence of all (a named unary operator has the highest). Use parentheses.

```
program test_dot
  use geometry
  implicit none
  type(vector2d) :: a, b
  a = vector2d(1.0, 2.0)
  b = vector2d(3.0, 4.0)
  print *, a .dot. b            ! 11.0
  print *, 1.0 + (a .dot. b)    ! 12.0
  ! Not allowed: read as (1.0 + a) .dot. b
  ! print *, 1.0 + a .dot. b
end program test_dot
```

The full module, with `+`, `-`, `*`, `/`, and `.dot.`, is in `examples/ex07_operators`.

Exercises: a star catalogue

Tasks

1. Write a module with a type `vector3d` and a type `star` that holds a name, a mass, and a position (a `vector3d`).
2. In a main program, create an array of three stars with constructors, and print the name of each star.
3. Print the total mass of the stars. Hint: `stars%mass` is an array.
4. Add a function `centre_of_mass(stars)` to the module that returns a `vector3d`, and test it.
5. Try to print one whole `star` with `print *`. Then make the name `character(len=:)`, `allocatable` and try again. What changes?

Exercises: vector operators

Tasks

1. Write a module with a type `vector3d` and the operators + and − for two vectors, and * for a real times a vector.
2. Add the named operators `.dot.` and `.cross.` for the scalar and vector products.
3. Test them: compute `c = a .cross. b` for two vectors of your choice, and check that `c .dot. a` and `c .dot. b` are both zero.
4. What does `2.0 * a .dot. b` compute? And `1.0 + a .dot. b`? Explain both.
5. Make all the operator functions `pure`. Why is that a sensible default?